

# StaleyNET: Text-Conditioned Music Generation Trained from Scratch on Edge GPUs in Rust

Arpan Pathak

November 2026

**Abstract**

StaleyNET generates music, including singing, from a short text description and, optionally, lyrics. We treat generation as probabilistic modeling. A recording is converted into a sequence of discrete audio tokens; the probability of that sequence given the text is factorized by the chain rule into a product of next-token predictions; and a decoder-only Transformer learns these conditional distributions by maximum likelihood. Its autoregressive structure is used twice: in training, a causal mask lets every factor of a sequence be evaluated in one pass, and in generation, the same factors are sampled one step at a time. Every stage is our own and written in Rust, including the spectrogram, the tokenizer, the hand-derived gradients and the GPU kernels, and the models are trained from random initialization on two NVIDIA Jetson devices with about 23 hours of Creative Commons music. Because the factorization separates the audio representation from the model of it, the error of the whole system can be decomposed stage by stage. This decomposition locates the main limitation in the tokenizer rather than in the language model: a model trained on a single excerpt reproduces its tokens exactly, while the tokens alone lose much of the spectral detail. A tokenizer that quantizes groups of frequency bands with more codes per step halves the reconstruction error. We also describe a paired test of whether a model uses its lyrics, which our first lyrics-conditioned models do not yet pass. Every algorithm is presented with a worked example.

## 1 Introduction

Text-to-music generation has advanced quickly [1, 3]. Most current systems assemble large components trained separately: a neural codec compresses audio into discrete *tokens*, small integers that each stand for a short stretch of sound; a text encoder turns the prompt into vectors; and a Transformer learns to predict the tokens from the text. Such a division of labor performs well, yet it obscures which component is responsible for a given quality, or a given flaw, in what one hears.

StaleyNET approaches the problem from the other end (Figure 1). Rather than borrowing pretrained parts, it builds every stage itself: it computes its own spectrograms, learns its own

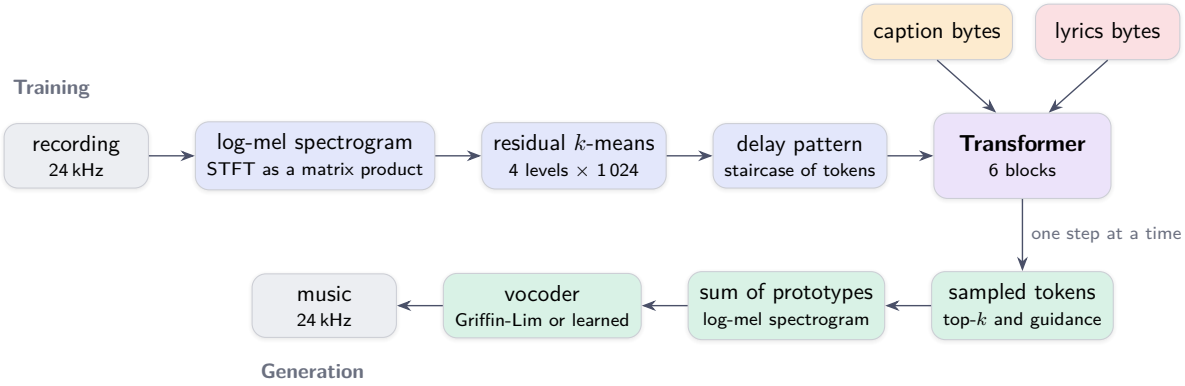



Figure 1: An overview of StaleyNET. During training (top row), a recording is converted into a staircase of discrete tokens, which the Transformer learns to predict from the caption and the lyrics. During generation (bottom row), the Transformer produces tokens one step at a time, and the tokens are turned back into sound.

tokenizer, trains its own Transformer and reconstructs its own waveforms. Everything is written in Rust, including the GPU kernels, which cuda-oxide [22] compiles from Rust. Training ran on two embedded computers designed for robotics: an NVIDIA Jetson Orin NX with 16 GB of memory and an NVIDIA Jetson AGX Thor with 128 GB.

A single probabilistic model organizes the whole system. Music is represented as a sequence of tokens, and its probability given the text is factorized by the chain rule into a product of next-token predictions (Section 5.1). One autoregressive Transformer learns all of these predictions at once, and generation samples them one after another. Because this factorization separates the audio representation from the model of it, each stage can be measured on its own, and much of this paper reports what those measurements revealed.

Our first models learned text-conditioned structure well above chance, yet their voices sounded blurred. Two experiments located the cause. Passed through the tokenizer and the vocoder alone, with no model involved, held-out recordings had already lost most of their spectral detail; trained on a single excerpt, the model reproduced it token for token. The tokenizer, not the language model, was the main limitation, and a revised tokenizer that quantizes groups of frequency bands separately halved the reconstruction error.

This paper makes four contributions.

- A complete text-to-music system written in Rust, GPU kernels included, and trained from scratch on embedded GPUs.
- Every algorithm stated as numbered pseudocode with a worked example and, where an algorithm is easy to get wrong, a proof of its correctness.
- A position map that adds a lyrics input to a trained model without discarding what it has learned, and a paired test of whether that input is used.
- A stage-by-stage error analysis that separates the tokenizer, the vocoder and the language model, and a tokenizer study guided by it.

**Conventions.** Algorithms follow the style of Cormen et al. [4], as in our tutorial on tiled matrix multiplication [23]: each procedure states its inputs and its output in words, with units, and the text refers to its numbered lines. $[a, b)$ denotes the integers $a, \ldots, b-1$, and indices start at 0. Each symbol is defined where it first appears, and Table 1 collects those used in more than one section. A few letters take a different meaning inside a single subsection, such as

$K$ for the keys of attention, and the text points this out.

Two units recur. Frequencies and sample rates are measured in hertz (Hz): a 440 Hz tone vibrates 440 times per second, and audio sampled at 24 kHz is stored as 24 000 numbers per second. Losses are measured in *nats*, the unit of information based on the natural logarithm: a prediction that assigns probability $q$ to the correct answer costs $-\ln q$ nats, so a certain prediction costs nothing and a uniform guess among 1 024 tokens costs $\ln 1024 \approx 6.93$.

Table 1: Symbols used in more than one section.

| symbol | meaning | value |
|---|---|---|
| $f_s$ | sample rate of the analyzed audio (samples per second) | 24 000 Hz |
| $N$ | samples in one spectrogram frame | 1 024 (42.7 ms) |
| $H$ | hop: samples between the starts of two frames | 256 (10.7 ms) |
| $S$ | log-mel spectrogram, one row per frame, one column per mel band | $\cdot \times 128$ |
| step | the frames described by one set of codes | 1 or 2 frames |
| $C$ | codes per step, one from each codebook | 4 or 16 |
| $K$ | prototypes per codebook (a codebook is a learned list of typical steps) | 1 024 |
| $G$ | groups of mel bands quantized separately | 1 or 4 |
| $d$ | width of the vectors inside the model | 256 or 512 |
| $L$ | bytes reserved for lyrics | 256 |
| $\theta$ | all trainable parameters of the model | 7.1 M or 42.9 M |

## 2 Related work

AudioLM [2] showed that audio can be generated by a language model over discrete tokens. MusicLM [1] conditioned this approach on text, and MusicGen [3] simplified it to a single Transformer over the codebooks of a neural codec, interleaved by a delay pattern. We follow MusicGen in using the delay pattern, top-$k$ sampling [10] and classifier-free guidance [13], and we replace its pretrained components with $k$-means quantization of log-mel frames and with raw text bytes, as studied for text by ByT5 [34].

Jukebox [8] generated singing from lyrics and learned the correspondence between words and sound from 1.2 million songs. Singing synthesis systems such as DiffSinger [17] instead work from a musical score with phoneme durations. Our lyrics experiments rely on the line-level timings of the JamendoLyrics benchmark [30, 9]. Neural codecs [36, 7, 16] and neural vocoders [15, 27] achieve far higher fidelity than $k$-means tokens and the Griffin-Lim algorithm [11, 24]. A codec costs more training than our setting allows; Section 9.1 measures what our simpler tokens cost, and Section 3.5 adapts Vocos to replace Griffin-Lim.

## 3 From sound to spectrogram and back

A model that generates music needs some description of sound, and the most direct one, the waveform, is a poor choice. It lists the displacement of the loudspeaker 24 000 times per second, and much of that detail is the exact *phase* of each vibration, where in its cycle it happens to be, which the ear barely notices. A model trained on waveforms would spend most of its capacity on information that listeners do not hear. StaleyNET instead describes sound by its *log-mel spectrogram*: a picture of how much energy each frequency band holds in each short moment of time. This section follows a recording on its way to such a picture, and a generated picture on its way back to sound.

### 3.1 From a recording to a clean signal

Training recordings arrive as MP3, FLAC, AAC, Ogg Vorbis or WAV files, at various sample rates and often in stereo. Each file is decoded into a list of *samples*, numbers between $-1$ and 1 that give the loudspeaker's displacement at successive instants, and stereo channels are averaged into one. The *sample rate*, the number of samples per second, is then unified at 24 kHz, so that a second of music always yields the same number of analysis frames. Last, every recording is scaled to the same average loudness (a root-mean-square level of $-20$ dBFS); otherwise the model would have to learn the large differences in level between commercial and amateur recordings as if they were part of the music.

Changing the sample rate is a filtering problem. A waveform sampled $f$ times per second can represent only frequencies below $f/2$, its *Nyquist frequency*, because at least two samples per cycle are needed to distinguish a vibration from silence. Lowering the rate to 24 kHz therefore requires first removing everything above 12 kHz; otherwise that content folds back into the audible range as false tones, an effect called *aliasing*.

**Definition 3.1** (Bandlimited resampling)**.** Let $x_m$ be sample $m$ of a recording made at $f_{\text{in}}$ samples per second, and let $f_{\text{out}}$ be the desired rate, also in samples per second. Output sample $n$ describes the instant $n/f_{\text{out}}$ seconds, which lies

$$t_n = n\,\frac{f_{\text{in}}}{f_{\text{out}}}$$

input samples after the start; $t_n$ is usually not a whole number. The resampled signal is

$$y_n \;=\; \sum_m x_m\, h(t_n - m)\,,$$

where the *filter* $h$ gives the weight of an input sample as a function of its distance $t_n - m$ from $t_n$, measured in input samples. Each output sample is therefore a weighted sum of the input samples near $t_n$.

Ideal weights come from $\operatorname{sinc}(u) = \sin(\pi u)/(\pi u)$, which interpolates a bandlimited signal exactly but extends infinitely far in both directions [29]. In practice it is cut short smoothly:

$$h(t) = c\,\operatorname{sinc}(ct)\,w(ct/Z).$$

Three parameters shape this filter. The *cutoff* $c \in (0, 1]$ is the highest frequency kept, as a fraction of the input Nyquist frequency; when the rate is lowered, it lies just below $f_{\text{out}}/f_{\text{in}}$. $Z = 24$ is the number of zero crossings of the sinc kept on each side, which sets how many input samples feed each output sample. And $w$ is a Kaiser window, a bell-shaped taper that reaches zero at $\pm 1$ so that truncating the sinc causes no ripples; its shape parameter $\beta = 8.6$ trades a sharper cutoff against stronger suppression of the removed frequencies. Evaluating $h$ afresh for every output sample would be slow, but the fractional part of $t_n$ takes only a few distinct values.

**A worked example.** Converting from $f_{\text{in}} = 44\,100$ Hz (the rate of a CD) to $f_{\text{out}} = 24\,000$ Hz, the two rates share the greatest common divisor 300, so $t_n = n \cdot 44\,100/24\,000 = 147n/80$ input samples. For $n = 1$, $t_1 = 1 + 67/80$: the second output sample lies 67/80 of the way from input sample 1 to input sample 2. Every $t_n$ has a fractional part $p/80$ with $p \in [0, 80)$, so exactly 80 sets of filter weights suffice. These sets are the *phases* of a polyphase filter, and each is computed once.

**RESAMPLE**$(x, f_{\text{in}}, f_{\text{out}})$

*Input:* the samples $x_0, x_1, \ldots$ of a recording (numbers between $-1$ and 1), the rate $f_{\text{in}}$ at which they were recorded, and the desired rate $f_{\text{out}}$; both rates are whole numbers of samples per second (Hz).
*Output:* the samples $y_0, y_1, \ldots$ of the same sound at $f_{\text{out}}$ samples per second; one second of input gives $f_{\text{out}}$ output samples.

```
1   g = gcd(f_in, f_out)
2   L = f_out / g // number of phases
3   M = f_in / g // input step per output sample, in L-ths
4   c = 0.97 · min(1, f_out / f_in) // cutoff
5   R = ⌈Z / c⌉ // filter half-width
6   for p ∈ [0, L) {
7       for j ∈ [−R, R] {
8           h_p[j] = c sinc(c (j − p/L)) w(c (j − p/L) / Z)
9       }
10  }
11  for each output index n {
12      m = ⌊nM / L⌋ // whole part of t_n
13      p = nM mod L // fraction of t_n
14      y_n = Σ_{j∈[−R,R]} h_p[j] x_{m+j}
15  }
```

Lines 1–5 fix the geometry of the filter: $L$ is the number of phases, $M/L = f_{\text{in}}/f_{\text{out}}$ is the distance between consecutive output samples in input samples, and $R$ is how many input samples are used on each side of $t_n$, which grows as $c$ shrinks because a lower cutoff needs a wider filter. Lines 6–10 compute each phase once, a list of $2R + 1$ weights, after which line 14 costs one short dot product per output sample. Setting the cutoff at 97 % of the new Nyquist frequency (line 4) leaves the filter a narrow band in which to fall from full strength to zero before aliasing could begin. A test confirms the result: converting 48 kHz audio to 24 kHz attenuates a 15 kHz tone, which the new rate cannot represent, by more than 80 dB, a factor of $10^8$ in power.

### 3.2 Measuring energy over time and frequency

A spectrogram cuts the signal into short, overlapping *frames* and measures the strength of each frequency within each frame. Four choices fix how fine the resulting picture is (Table 2).

Table 2: Analysis settings and what each one means.

| setting | value | meaning |
|---|---|---|
| frame length | $N = 1\,024$ samples (42.7 ms), Hann window | long enough to separate frequencies 23.4 Hz apart; the window fades each frame in and out so its edges create no spurious frequencies |
| hop | $H = 256$ samples (10.7 ms) | 93.75 frames per second, each overlapping the next by three quarters |
| frequency bands | 128 mel bands, 0 to 12 kHz | narrow at low frequencies and wide at high ones, matching the ear's resolution [28] |
| compression | natural logarithm, floor $10^{-5}$ | loudness is perceived roughly logarithmically; the floor keeps silence finite |

Speech systems usually use 80 to 100 bands; music, with harmonic detail across a wider range, benefits from 128. One second of audio becomes a grid of about 94 frames by 128 bands,

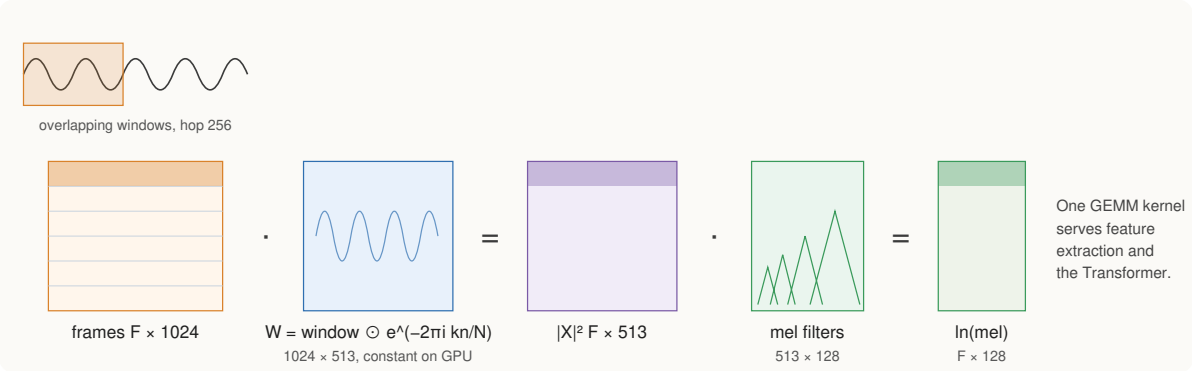



Figure 2: The spectrogram computed as two matrix products. The frames of the signal, one per row, are multiplied by the windowed Fourier matrix $W$, which gives the energy $|X|^2$ at each of 513 frequencies; a second product with the mel filters pools these into 128 bands, and the logarithm compresses them. The same matrix-product kernel serves the spectrogram and the Transformer.

a far more compact description than 24 000 samples.

### 3.3 The spectrogram as a matrix product

Frequencies are usually measured with the fast Fourier transform (FFT). A Fourier transform can also be written as an ordinary matrix product: each frequency is a fixed pattern of sines and cosines, and multiplying a frame by the matrix that holds these patterns measures how strongly each one is present. Written this way, analyzing a whole recording is one large matrix product, and pooling frequencies into mel bands is a second (Figure 2). StaleyNET computes its spectrograms this way, so that they run on the same GPU kernel as the Transformer and inherit its tests (Section 8).

**Definition 3.2** (Log-mel spectrogram)**.** Let $F$ be the matrix whose row $f$ holds the $N$ samples of frame $f$, and let $W$ be the $N \times (N/2+1)$ matrix with entries $W_{nk} = w[n]\, e^{-2\pi i k n/N}$, where $n \in [0, N)$ indexes the samples of a frame, $k \in [0, N/2]$ indexes the frequencies, $w[n] = \sin^2(\pi n/N)$ is the Hann window and $i = \sqrt{-1}$. The log-mel spectrogram is

$$S \;=\; \ln \max\bigl(|FW|^2 B,\; 10^{-5}\bigr),$$

where the absolute value and the square are taken entry by entry, and $B$ is the $(N/2+1) \times 128$ matrix whose columns are 128 triangular mel filters [28]. $S$ has one row per frame and one column per mel band, and its entries are natural logarithms of energy.

Column $k$ of $W$ holds a windowed cosine (real part) and sine (imaginary part) at frequency $k f_s/N = k \cdot 23.4\,\text{Hz}$, so entry $(f, k)$ of $FW$ measures how much of that frequency frame $f$ contains, and $|FW|^2$ is its energy. The mel filters then pool the 513 frequencies into 128 bands. Like pitch perception, the *mel* scale is linear below 1 kHz and logarithmic above: centers of neighboring bands lie about 26 Hz apart at low frequencies and about 330 Hz apart near 12 kHz, much as the ear tells low notes apart more finely than high ones.

This formulation costs about $N/\log_2 N \approx 100$ times the arithmetic of an FFT, because a matrix product computes every frequency separately while the FFT shares work between them. Even so, on the Orin NX it analyzes 60 s of audio in 47 ms, against 73 ms for a multithreaded CPU FFT (Table 11), and the two agree to within $10^{-2}$ log units in every audible band.

### 3.4 Turning a spectrogram back into sound

After generation, a spectrogram must become a waveform again, and this direction is harder: a spectrogram records how strong each frequency is but not its phase. The phase has to be estimated, and not every combination of magnitudes and phases belongs to a real waveform, because neighboring frames overlap by three quarters and must agree about the samples they share. The Griffin-Lim algorithm [11] searches for a phase by alternating between two requirements: the magnitudes should match the target, and the spectrogram should be that of an actual waveform (it should be *consistent*).

**GRIFFIN-LIM**( $|S|, T$ )

*Input:* the target magnitudes $|S|$, a matrix of non-negative numbers with one row per frame and one column for each of the $N/2 + 1 = 513$ frequencies, and the number of rounds $T$.
*Output:* a waveform $x$ at $f_s$ samples per second whose spectrogram approximates $|S|$.

1 $X = |S| \cdot e^{i\phi}$ with random phases $\phi$
2 $x = \mathrm{ISTFT}(X)$
3 $P_{\mathrm{prev}} = \mathrm{STFT}(x)$
4 **for** $t \in [0, T)$ {
5 $P = \mathrm{STFT}(x)$ *// nearest consistent spectrogram*
6 $A = P + \alpha\,(P - P_{\mathrm{prev}})$ *// momentum*
7 $X = |S| \cdot A/|A|$ *// keep the phase, restore the magnitude*
8 $P_{\mathrm{prev}} = P$
9 $x = \mathrm{ISTFT}(X)$
10 }
11 **return** $x$

STFT (the short-time Fourier transform) turns a waveform into its complex spectrogram, $FW$ above, and ISTFT turns a complex spectrogram back into a waveform by overlapping and adding its frames. Every entry of a complex spectrogram is a magnitude times $e^{i\phi}$, with $\phi$ the phase in radians. Line 1 pairs the target magnitudes with random phases. Line 5 replaces the estimate by the spectrogram $P$ of a real waveform, and line 7 keeps the phase of each entry while restoring its target magnitude. Line 6 is the acceleration of Perraudin et al. [24], which extrapolates along the last change with momentum $\alpha = 0.99$; in the first round it has no effect, because $P = P_{\mathrm{prev}}$. We use $T = 64$ rounds. Since $S$ holds logarithms of mel energies, the target magnitudes come from undoing its definition step by step: $e^S$ gives the band energies, a regularized least-squares inverse of $B$ spreads them back over the 513 frequencies, and a square root turns energy into magnitude.

**Limits of Griffin-Lim.** The algorithm needs no training, and it recovers the magnitudes well, but its phase remains an estimate, and that is audible. A sustained harmonic, a sung vowel for example, needs a phase that advances smoothly from frame to frame at the harmonic's own frequency. Griffin-Lim only makes neighboring frames agree, so its phases drift and partly cancel where frames overlap, which a listener hears as a metallic, phasey smear. The smear is worst on voices, whose pitch and vowel identity live in exactly these harmonics, and on sharp onsets, which spread into a faint pre-echo. Even on a perfect spectrogram the method adds a spectral error of 0.19 log units per band (Table 5), and the artifacts it causes are larger than that number suggests, because a spectral error measures magnitudes only. A neural vocoder, which learns phase from real recordings [15, 27], avoids these artifacts; Section 3.5 describes

ours.

### 3.5 A learned vocoder

Our vocoder follows Vocos [27]. Many neural vocoders generate the waveform sample by sample, or upsample it through a stack of transposed convolutions; Vocos instead stays at the frame rate, predicts the magnitude and the phase of every frequency of every frame, and leaves the step to 24 000 samples per second to an inverse STFT (Figure 3). The network therefore runs on 94 frames per second instead of 24 000 samples, and the inverse transform is the same pair of matrix products as the analysis, read in the other direction.

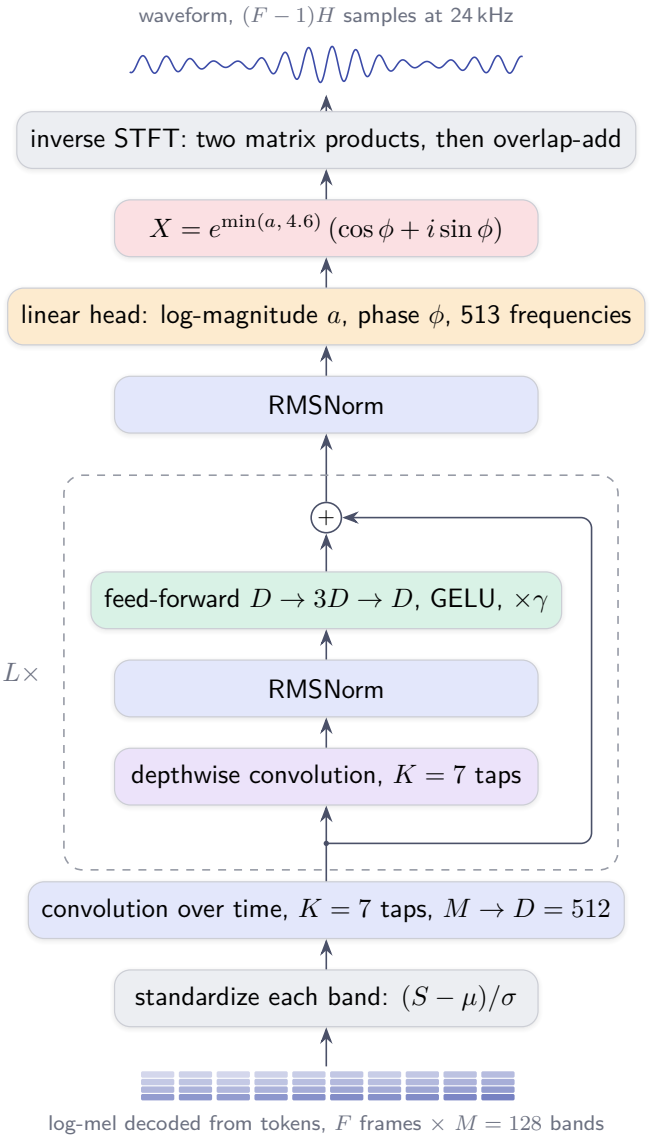



Figure 3: The learned vocoder. A convolution lifts each frame's mel bands to $D$ numbers, $L$ ConvNeXt blocks mix neighboring frames, and a head predicts the log-magnitude and phase of every frequency. The inverse STFT turns that spectrum into samples with two matrix products and overlap-add.

**Input.** The input is a log-mel spectrogram $S$ with $F$ frames (rows) and $M = 128$ bands (columns), decoded from tokens. Each band is first standardized with the tokenizer's statistics,

$\hat{S}_{f,m} = (S_{f,m} - \mu_m)/\sigma_m$, where $\mu_m$ and $\sigma_m$ are the mean and standard deviation of band $m$ over the training recordings, so that every band enters the network on a similar scale.

**Network.** A convolution over time with $K = 7$ taps maps the $M$ values of each frame and of its three neighbors on either side to a vector of $D = 512$ numbers. We compute it as one matrix product, after copying each frame's neighborhood into a row of $KM$ numbers. A stack of $L = 8$ ConvNeXt blocks [18] follows. With $h$ the $F \times D$ matrix of frame vectors entering a block, the block computes

$$h \leftarrow h + \gamma \odot (\mathrm{GELU}(\mathrm{Norm}(\mathrm{DW}(h))\, W_1 + b_1)\, W_2 + b_2),$$

where DW filters each of the $D$ channels over time with its own $K$ taps (a *depthwise* convolution, which mixes neighboring frames but not channels), Norm is RMS normalization [37], $W_1$ is $D \times 3D$ and $W_2$ is $3D \times D$, so that the feed-forward part mixes channels within a frame, $b_1$ and $b_2$ are biases, and $\gamma$ is a vector of $D$ per-channel scales, initialized to $1/L$ so that each block starts as a small correction to its input. A final normalization and a linear map, $D \times 2(N/2+1)$, give each frame a log-magnitude $a_{f,k}$ and a phase $\phi_{f,k}$ (in radians) for each of the 513 frequencies $k$. The complex spectrum is

$$X_{f,k} = e^{\min(a_{f,k},\, a_{\max})}\, (\cos\phi_{f,k} + i \sin\phi_{f,k}),$$

with $a_{\max} = 4.6$, so that no magnitude exceeds $e^{4.6} \approx 100$ and an untrained network cannot overflow. The network has 13.6 million parameters.

**Inverse STFT.** Frame $f$ becomes $N$ samples through the inverse Fourier transform of a real signal,

$$y_f[n] = \frac{w[n]}{N} \sum_{k=0}^{N/2} c_k (\mathrm{Re}\, X_{f,k} \cos\tfrac{2\pi kn}{N} - \mathrm{Im}\, X_{f,k} \sin\tfrac{2\pi kn}{N}),$$

where $n \in [0, N)$ is the sample within the frame, $w$ is the Hann window, and $c_k = 1$ for $k = 0$ and $k = N/2$ and 2 otherwise, since every other frequency stands for itself and its mirror image. For all frames at once this is two matrix products, $(F \times 513) \cdot (513 \times N)$, with the cosines and sines, the window and $c_k/N$ folded into constant matrices. Overlap-add then places frame $f$ at samples $fH$ to $fH + N - 1$ and divides each sample by the sum of squared window values that cover it, which makes analysis followed by synthesis return the original signal [11].

**Loss.** The vocoder is trained to make its waveform $\hat{x}$ sound like the recording $x$ that the spectrogram came from, measured in the frequency domain at several resolutions, as in Parallel WaveGAN [35]. For a resolution with frame length $N_r$ and hop $H_r$, let $E_r(x)$ be the matrix of energies of $x$: the squared magnitudes of its STFT, pooled into mel bands for one of the resolutions. The loss is

$$\mathcal{L} = \sum_r \lambda_r\, \mathrm{mean} \big| \ln \max(E_r(\hat{x}), \varepsilon) - \ln \max(E_r(x), \varepsilon) \big|,$$

where the mean runs over all frames and frequencies (or bands), $\varepsilon = 10^{-5}$ is the floor of Table 2, and the three resolutions are log-mel energies at $(N_r, H_r) = (1024, 256)$ with weight $\lambda_r = 1$, and log energies at $(512, 128)$ and $(2048, 256)$ with weight 0.5 each. The short frames reward sharp timing, the long ones fine frequency, and the mel term weighs errors roughly as the ear does [15]. Vocos also trains against learned discriminators; we begin with these spectral terms alone, which are differentiable in closed form and stable.

**Gradients.** As for the language model, every gradient is written by hand. One observation saves most of the work: cutting a signal into overlapping frames and overlap-adding frames into a signal are adjoint linear maps, so the gradient of each is the other. A single pair of kernels therefore serves the inverse STFT, its gradient, the framing inside every loss and that framing's gradient. The gradient of the absolute value is the sign of its argument, and the gradient of $e^a(\cos\phi + i\sin\phi)$ with respect to $a$ and $\phi$ follows from the chain rule. A test perturbs parameters of every kind, one at a time, and compares the change of the loss with the computed gradient; the two agree to within 5%.

**Training data.** The vocoder only ever sees spectrograms decoded from tokens, so it is trained on exactly those. The data are 32 882 clips of 30 s (273 hours) of rock and its subgenres from the Free Music Archive [6], tokenized with the tokenizer of Section 4. For every clip we store its samples, resampled and level-normalized as for analysis. A training example is a window of $T = 64$ consecutive frames decoded from the recording's tokens, together with the $(T-1)H + N = 17\,152$ samples (0.71 s) that those frames cover, since frame $f$ is centered on sample $fH$. The vocoder thus learns the phase and also part of the detail that the tokens lose. A batch holds 16 windows from random recordings. Training uses AdamW [20] with $\beta_1 = 0.8$, $\beta_2 = 0.9$, weight decay 0.01 and gradients clipped to norm 1; the learning rate rises linearly to $5 \cdot 10^{-4}$ over 1 000 steps and then follows a cosine down to $5 \cdot 10^{-5}$ at 200 000 steps.

# 4 From spectrogram to tokens

A language model predicts the next item of a sequence by choosing from a fixed list of possibilities, its *vocabulary*, just as a text model chooses the next word. A spectrogram, however, consists of real numbers that can take infinitely many values. Before a language model can learn from audio, each moment of the spectrogram must be replaced by choices from a finite list. This replacement is *tokenization*, and its result is a sequence of *tokens*, small integers that each stand for a piece of sound. Large systems use a pretrained neural codec for this purpose [36, 7, 16]. StaleyNET builds its own tokenizer, simple enough to train in minutes: $k$-means clustering applied in several rounds, known as residual vector quantization.

**What is tokenized.** Every band of the spectrogram is first standardized: its mean over the training set is subtracted and the result is divided by its standard deviation, so that quiet and loud bands weigh alike and every value is a plain number without unit. Consecutive frames are then grouped into *steps*, the unit that receives tokens. In our first tokenizer a step joins 2 frames, a vector of $D = 2 \cdot 128 = 256$ numbers covering $2H/f_\mathrm{s} = 21.3$ ms of sound; a two-minute song becomes about 5 600 steps.

## 4.1 Residual quantization

Suppose a collection of $K = 1\,024$ typical steps has been found. We call them *prototypes*, and the list that holds them a *codebook*. Any new step can then be described by naming the prototype that most resembles it: 256 real numbers become one integer between 0 and 1 023, the step's *code*. Lloyd's $k$-means algorithm [19] finds such prototypes, by alternately assigning every training step to its nearest prototype and moving each prototype to the average of its steps.

One codebook captures only the broad shape of a step. The difference between the step and its prototype, the *residual*, still holds useful detail, so residual quantization repeats the procedure on it: a second codebook is trained on the errors left by the first, a third on those left

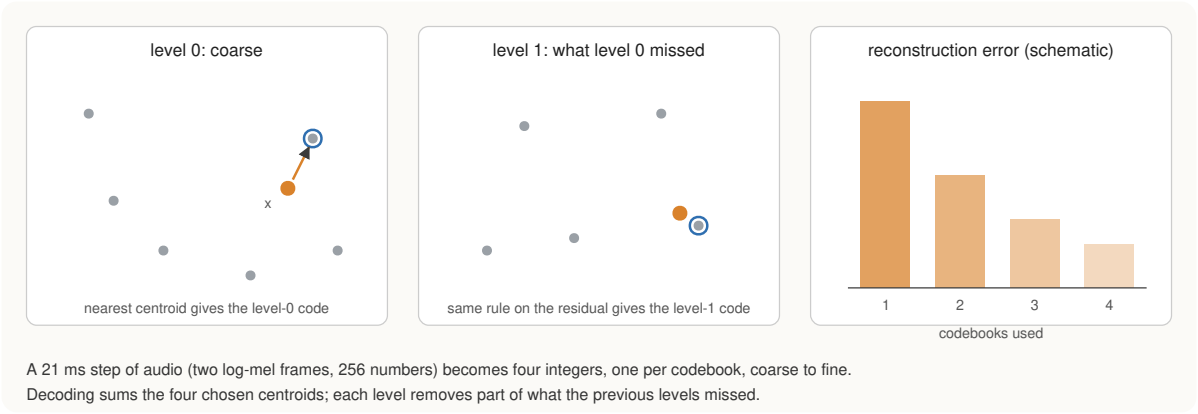



Figure 4: Residual quantization as a picture. Level 0 replaces a step $x$ by the nearest of its prototypes (left); level 1 applies the same rule to what level 0 missed (middle); each further level removes part of the error that remains (right, schematic). The step is then described by one code per level.

by the second, and so on. The idea resembles describing a color by its family, then its shade within the family, then finer and finer corrections. Each round, or *level*, contributes one code, so a step is described by one code per level, ordered from coarse to fine (Figure 4).

**Definition 4.1** (Residual vector quantization)**.** Let $E_0, \dots, E_{C-1}$ be $C$ codebooks (one per level $\ell$), each a list of $K$ prototype vectors of length $D$, and write $E_\ell[k]$ for prototype $k$ of codebook $\ell$. A step $x \in \mathbb{R}^D$ is encoded as the codes $c_0, \dots, c_{C-1}$ defined by

$$r_0 = x, \qquad c_\ell = \arg\min_{k \in [0,K)} \|r_\ell - E_\ell[k]\|^2,$$
$$r_{\ell+1} = r_\ell - E_\ell[c_\ell],$$

and it is decoded as $\hat{x} = \sum_\ell E_\ell[c_\ell]$. The vector $r_\ell$ is the *residual*: the part of $x$ that levels $0, \dots, \ell - 1$ have not yet described.

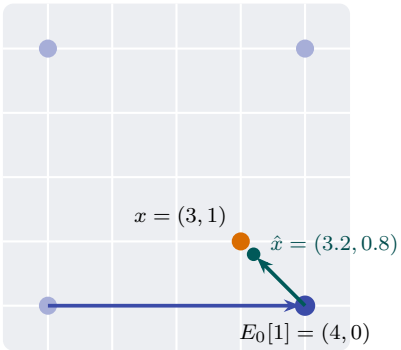



Figure 5: Residual quantization of $x = (3, 1)$ in two levels. Level 0 (blue) chooses the prototype $(4, 0)$, and level 1 (green) adds the correction $(-0.8, 0.8)$.

**A worked example.** In two dimensions, consider the codebooks

$$E_0 = \{(0,0),\ (4,0),\ (0,4),\ (4,4)\},$$
$$E_1 = \{(0.8,0.8),\ (-0.8,0.8),\ (-0.8,-0.8),\ (0.8,-0.8)\},$$

and the point $x = (3, 1)$ (Figure 5). Its squared distances to the prototypes of $E_0$ are $10, 2, 18, 10$, so level 0 picks $c_0 = 1$, the prototype $(4, 0)$, and leaves the residual $r_1 = (-1, 1)$. Of the corrections in $E_1$, $(-0.8, 0.8)$ lies closest to that residual, so $c_1 = 1$ and only $r_2 = (-0.2, 0.2)$ remains. Two small integers now describe the point, and its error falls from $\sqrt{2} \approx 1.41$ after one level to $\sqrt{0.08} \approx 0.28$ after two.

## 4.2 Training the codebooks

Codebooks are trained level by level. Each level runs Lloyd's algorithm on what the levels before it left unexplained, then hands its own residuals to the next.

**TRAIN-CODEBOOKS**$(X, C, K, I)$

*Input:* a matrix $X$ with one row per training step and $D$ standardized values per row, the number of levels $C$, the number of prototypes per level $K$, and the number of refinement rounds $I$.
*Output:* the codebooks $E_0, \ldots, E_{C-1}$, each a $K \times D$ matrix whose rows are prototypes.

```
1   R = X // what remains to be described
2   for ℓ ∈ [0, C) {
3     E_ℓ = K rows of R, chosen at random
4     repeat I times {
5       a_i = NEAREST(R_i, E_ℓ) for each row i
6       E_ℓ[k] = mean{R_i : a_i = k} for each k
7       if no row has a_i = k
8         E_ℓ[k] = a random row of R
9     }
10    R_i = R_i − E_ℓ[NEAREST(R_i, E_ℓ)] for each row i
11  }
```

**NEAREST**$(r, E)$ returns the index of the prototype in $E$ closest to $r$ in Euclidean distance. $R$ holds the residuals of all training steps, one row per step, and $a_i \in [0, K)$ records which prototype row $i$ is currently assigned to. Lines 5–6 are the two moves of Lloyd's algorithm [19]: each row is assigned to its nearest prototype, and each prototype then moves to the mean of its rows. Lines 7–8 restart a prototype to which no row was assigned, so that none of the $K$ entries goes unused, and line 10 passes the residuals on to the next level. We use $I = 20$ rounds on 200 000 steps drawn from the training set.

## 4.3 The nearest prototype as a matrix product

Finding the nearest prototype for every step is the most expensive part of both training and encoding. The following identity turns it into a matrix product, which the GPU computes efficiently.

**Lemma 4.1.** *For a vector $x$ and prototypes $e_0, \ldots, e_{K-1}$, all of length $D$,* $\arg\min_k \|x - e_k\|^2 = \arg\min_k \left(\|e_k\|^2 - 2\, x \cdot e_k\right)$.

*Proof.* Expanding gives $\|x - e_k\|^2 = \|x\|^2 - 2\, x \cdot e_k + \|e_k\|^2$, and the first term does not depend on $k$. □

In the example above, the squared norms $\|e_k\|^2$ are $0, 16, 16, 32$ and the dot products $x \cdot e_k$

are $0, 12, 4, 16$, so the scores are $0, -8, 8, 0$, and the smallest again selects $(4, 0)$. On the GPU the dot products of 8 192 steps with 1 024 prototypes form a single matrix product, after which a small kernel finds the smallest score in each row.

### 4.4 Quantizing groups of bands

The analysis of Section 9.1 showed that four levels over a whole step lose too much detail: 256 numbers are a great deal to describe with four choices among 1 024 prototypes. Our final tokenizer changes two things. A step becomes a single frame, 128 numbers covering 10.7 ms, so that each code describes a shorter moment. And the 128 bands are divided into $G = 4$ groups of 32 neighboring bands, from the lowest frequencies to the highest, each with its own residual codebooks. A group of 32 numbers is far easier to describe with 1 024 prototypes than a whole step: the same number of prototypes now covers a much smaller space, so each one lies closer to the vectors it stands for.

Each group is quantized in 4 levels, so a step is described by $C = 4 \times 4 = 16$ codes. They are ordered level by level: first the coarse code of every group, then their first corrections, and so on, which preserves the coarse-to-fine order on which the delay pattern relies (Section 5). The change halves the reconstruction error (Section 9.1). A control with 16 levels and no groups shows that most of that gain comes from the larger number of codes per step, and a smaller part from the grouping; Table 12 in the appendix compares all the configurations we tried.

## 5 The model

At the heart of StaleyNET is a Transformer of the kind used for language models such as GPT-2 [31, 26] (Figure 6): a stack of identical blocks that reads one long sequence, beginning with the text and continuing with the audio, and predicts at every position what comes next. It is called *decoder-only* because one stack both reads the text and produces the audio, with no separate encoder.

### 5.1 The probability model

The whole system rests on one modeling decision, which also explains the shape of the rest of this section. Let $\mathcal{T}$ be the conditioning text, the caption and, when present, the lyrics, given as a string of bytes. Let a piece of music be the sequence of its tokenizer steps $a_1, \dots, a_F$, where step $t$ consists of the $C$ codes $a_t^0, \dots, a_t^{C-1}$, ordered from coarse to fine. We model the probability of a piece given its text. By the chain rule of probability, any such distribution can be written, without approximation, as a product of predictions of one code at a time:

$$p(a_1, \dots, a_F \mid \mathcal{T}) \;=\; \prod_{t=1}^{F} \prod_{\ell=0}^{C-1} p(a_t^\ell \mid \mathcal{T},\, a_{<t},\, a_t^{<\ell}),$$

where $a_{<t}$ stands for all the codes of steps $1, \dots, t-1$ and $a_t^{<\ell}$ for the codes $a_t^0, \dots, a_t^{\ell-1}$ of step $t$ that are coarser than $a_t^\ell$. In words, the probability of a piece is the probability of its first code given the text, times the probability of the second given the text and the first, and so on. Each factor asks only what comes next, given everything before it. This is exactly the question that an autoregressive Transformer answers, and it turns a distribution over whole pieces, which no network could represent directly, into a sequence of classifications among $K = 1\,024$ codes.

The factorization determines both halves of the system. *Training* chooses the parameters $\theta$ that make the recorded pieces as probable as possible. Taking the logarithm turns the product

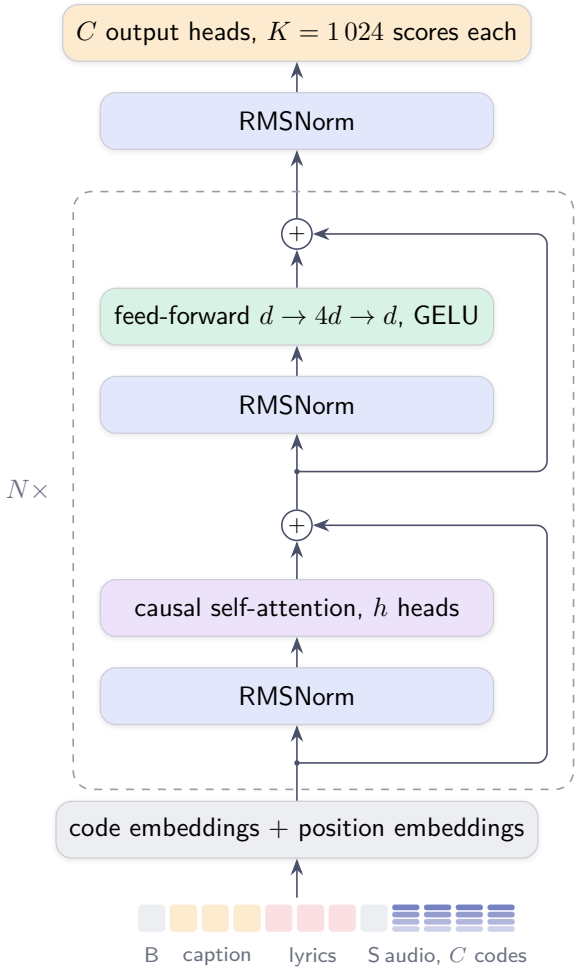



Figure 6: The model. Text bytes and audio tokens share one sequence, and each audio position carries all the codes of one step. A stack of pre-normalized blocks, each adding its result to its input, ends in one output head per code.

into a sum, so maximizing the likelihood is the same as minimizing the sum of $-\ln p_\theta$ over every code of every piece, which is the cross-entropy loss of Section 5.4. Because each factor depends only on earlier codes, all the factors of a training sequence can be evaluated in a single forward pass, with a causal mask that hides later positions from earlier ones. *Generation* follows the same product forward: it draws the first code from its predicted distribution, appends it to the sequence, draws the next, and so on, so that the finished piece is a sample of $p_\theta(\cdot \mid \mathcal{T})$; top-$k$ sampling and guidance (Section 7) adjust the distribution at each step.

Each factor conditions code $\ell$ of a step on the coarser codes of the same step, because a correction is easier to predict once the value it corrects is known. A model that predicted all $C$ codes of a step at one position would have to drop that conditioning. The delay pattern below restores it at a small cost.

## 5.2 One sequence for text and audio

A Transformer reads a sequence of *positions*, and each position holds one or more symbols from a shared vocabulary. StaleyNET's vocabulary combines three kinds of symbol: the 256 possible values of a byte, so that any text can be written directly as its UTF-8 bytes with no separate text encoder [34]; three markers, `BEGIN`, `SEPARATOR` and `PADDING`, which delimit the parts of the sequence; and $C \cdot K$ audio symbols, one for each prototype of each codebook. Text always fills 128 positions: `BEGIN`, up to 126 caption bytes (padded when shorter) and `SEPARATOR`. Keeping that length fixed makes the audio start at the same position in every sequence, which simplifies both training and generation.

Each audio position carries all $C$ codes of one step. A learned vector of $d$ numbers, its *embedding*, represents every symbol and every position, and an audio position enters the model as the sum of the embeddings of its $C$ codes and of the position itself.

## 5.3 The delay pattern

At each position the model predicts $C$ codes at once, one per output head. The factorization of Section 5.1, however, asks for each finer code only after the coarser codes of the same step are known.

**Definition 5.1** (Delay pattern [3])**.** Write $a_u^\ell$ for the code of codebook $\ell$ at step $u$. Position $t$ of the delayed sequence holds, in slot $\ell \in [0, C)$, the code $a_{t-\ell}^\ell$, so that codebook $\ell$ runs $\ell$ positions behind codebook 0, and an empty marker when $t - \ell$ is not a step of the piece. A piece of $F$ steps becomes $F + C - 1$ positions.

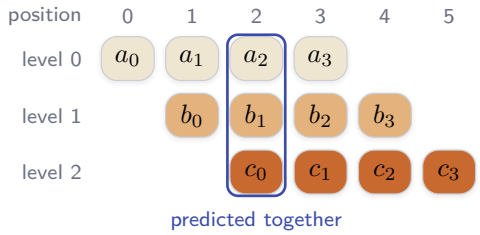



Figure 7: The delay pattern for three codebooks and four steps. The letters $a$, $b$ and $c$ stand for codebooks 0, 1 and 2, and the subscript is the step. At position 2 the model predicts $a_2$, $b_1$ and $c_0$, and the coarser codes that they refine are already part of its input.

In Figure 7, position 2 predicts $a_2$, $b_1$ and $c_0$ together. By then $a_1$, which $b_1$ refines, and $a_0$ and $b_0$, which $c_0$ refines, are already known. Code $\ell$ of every step is therefore predicted after

all coarser codes of its step, as the chain rule requires. The price is $C-1$ additional positions, whereas giving each codebook its own positions would multiply the length of the sequence by $C$.

### 5.4 The network and its objective

Each block does two things. *Attention* lets every position gather information from earlier positions: each position forms a query, compares it with a key formed by every earlier position, and takes a weighted average of their values, the weights reflecting how well query and key match. Attention is *causal*: a position may look only at itself and at earlier positions, since during generation later ones do not yet exist. A small *feed-forward* network then transforms each position on its own. Each part first rescales its input to a standard size (RMS normalization [37, 33]) and then adds its result to its input rather than replacing it, a *residual connection* that keeps deep stacks easy to train:

$$h = x + \mathrm{Attention}(\mathrm{Norm}(x)),$$
$$y = h + \mathrm{FeedForward}(\mathrm{Norm}(h)),$$

where $x$ is the sequence entering the block, one vector of $d$ numbers per position, and $y$ the sequence leaving it. Attention is split into $h$ *heads*, each attending with its own $d/h$ numbers, so that different heads can track different relations; the feed-forward network expands each vector to $4d$ numbers, applies the GELU activation [12], and projects it back to $d$. Table 3 lists the two models of this paper.

Table 3: Model sizes: blocks $N$, width $d$, heads $h$ and codes per step $C$.

| model | $N$ | $d$ | $h$ | $C$ | parameters |
|---|---|---|---|---|---|
| first | 6 | 256 | 8 | 4 | 7.1 M |
| large (Section 9.3) | 8 | 512 | 8 | 16 | 42.9 M |

At position $p$ the model outputs, for each codebook $\ell$, a score for each of the $K$ possible codes, and a softmax turns the scores into a probability distribution over the code $y_{p,\ell}$ that appears at position $p+1$. The loss is the cross-entropy, averaged first over the set $V_\ell$ of positions that have a target in codebook $\ell$ and then over the codebooks:

$$\mathcal{L} = \frac{1}{C}\sum_{\ell=0}^{C-1}\frac{1}{|V_\ell|}\sum_{p\in V_\ell} -\ln P_\theta(y_{p,\ell} \mid z_0,\dots,z_p),$$

where $z_p$ is the input at position $p$, $\theta$ denotes the parameters, and $P_\theta(y \mid \cdot)$ is the probability that the model assigns to the correct code $y$. This is the negative logarithm of the likelihood of Section 5.1, averaged per code, so minimizing it maximizes the probability of the training music. The loss is measured in nats per code. A model that assigns equal probability to all 1 024 codes has a loss of $\ln 1024 \approx 6.93$, and lower values mean that the model has learned something about the music.

### 5.5 Gradients by hand

As far as we know, no automatic differentiation is available for Rust on the GPU, so each layer implements its own gradient. Attention is the least obvious case. For one head, let the rows of $Q$, $K$ and $V$ be the queries, keys and values of the positions: each position *asks* with its query, the similarity of a query and a key decides how much attention a position pays to another, and the values are what is passed along. In this subsection $K$ and $V$ denote keys and values rather

than codebook sizes, and $d$ denotes the width of one head. With $T$ positions, $Q$, $K$ and $V$ are $T \times d$ matrices, and

$$S = QK^\top/\sqrt{d} + M, \qquad P = \mathrm{softmax}(S), \qquad O = PV,$$

where $S$, $P$ and the mask $M$ are $T \times T$, $O$ is $T \times d$, $M$ is $-\infty$ above the diagonal so that no position attends to a later one, and the softmax turns each row of scores into weights that are positive and sum to 1.

**Proposition 5.1.** *Let $\partial O$ be the gradient of the loss with respect to $O$, and similarly for the other matrices, and let $\delta_{jk}$ be 1 when $j = k$ and 0 otherwise. Then $\partial V = P^\top \partial O$, $\partial P = \partial O\, V^\top$, $\partial S_{ik} = P_{ik}(\partial P_{ik} - \rho_i)$ with $\rho_i = \sum_j \partial P_{ij} P_{ij}$, $\partial Q = \partial S\, K/\sqrt{d}$ and $\partial K = \partial S^\top Q/\sqrt{d}$.*

*Proof.* The first two identities follow from $O = PV$. Within a row, $\partial P_{ij}/\partial S_{ik} = P_{ij}(\delta_{jk} - P_{ik})$, so $\partial S_{ik} = \sum_j \partial P_{ij} P_{ij}(\delta_{jk} - P_{ik}) = P_{ik}(\partial P_{ik} - \rho_i)$. The last two follow from $S = QK^\top/\sqrt{d}$. Where $M = -\infty$ we have $P_{ik} = 0$ and therefore $\partial S_{ik} = 0$. □

Apart from $\partial S$, every gradient is a matrix product and runs on the same kernel as the forward pass. Because hand-written gradients are easy to get wrong, a test perturbs three random entries of every parameter tensor by $\pm 10^{-2}$ and checks that the resulting change of the loss agrees with the computed gradient to within 5 %.

# 6 Lyrics

A caption describes how a piece should sound, but not which words should be sung. To make the words controllable, the model needs a second text input, and it needs training examples in which it can hear which words are sung where.

## 6.1 A position map that preserves a trained model

The lyrics occupy $L = 256$ byte positions between the caption and the separator. Each position $p$ takes its position embedding from row $\pi(p)$ of a table of learned vectors, and the choice of the map $\pi$ determines whether a model trained without lyrics can later be extended (Figure 8). Simply inserting the lyrics would shift every later position, so the audio would read embeddings that the trained model had learned for other places.

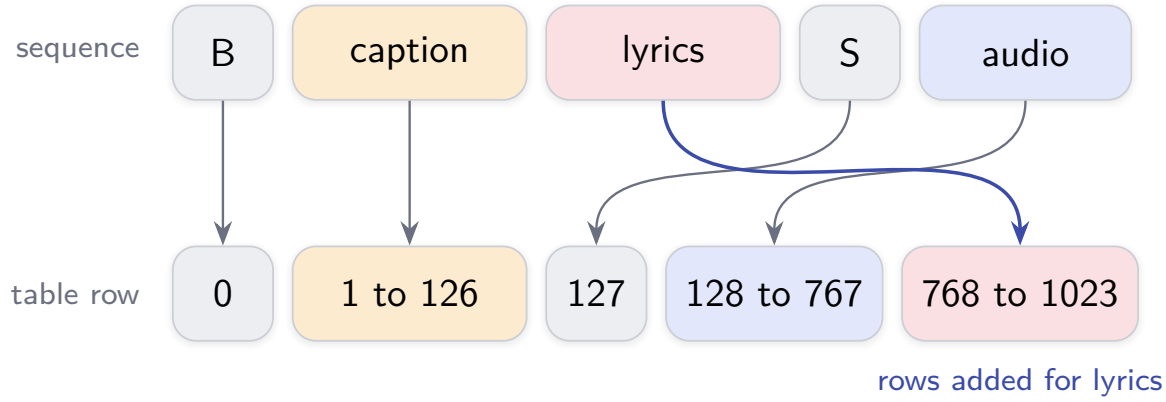



Figure 8: The position map $\pi$. Positions outside the lyrics keep the rows that they used before the lyrics were added, and the lyrics use new rows appended to the table.

**Definition 6.1** (Position map)**.** Suppose that position 0 holds `BEGIN`, the caption occupies positions $[1, 1{+}P)$, where $P$ is the number of caption bytes, the lyrics occupy $[1{+}P, 1{+}P{+}L)$, and the position table of the model without lyrics has $T$ rows, one for each position it can read. Then

$$\pi(p) = \begin{cases} p & \text{if } p < 1 + P, \\ T + p - 1 - P & \text{if } 1 + P \le p < 1 + P + L, \\ p - L & \text{otherwise.} \end{cases}$$

**Lemma 6.1.** *Every position outside the lyrics uses the row that the same part of the sequence used without lyrics, and the lyrics use only the new rows* $[T, T + L)$*.*

*Proof.* Caption positions are unchanged. The separator and the audio are shifted right by $L$ positions, and $\pi$ subtracts $L$. The lyrics are mapped onto $T + [0, L)$. □

For example, with $P = 2$, $L = 3$ and $T = 16$, positions 0 to 8 use the rows

$$0,\ 1,\ 2,\ 16,\ 17,\ 18,\ 3,\ 4,\ 5.$$

A trained model can therefore be extended by appending $L$ new rows to its table. All learned parameters keep their meaning, and training simply continues.

## 6.2 Pairing audio with the words sung in it

A training example covers a window of a song, and it should carry the lines sung in that window. JamendoLyrics [9] gives the start and end time of every line of 78 songs in English, French, German and Spanish, and a window, the stretch of a song shown to the model in one training example, receives each line whose time span overlaps it. Takes recorded with our studio come with their text but without timings, so we assume that their lines are spread evenly over the take; this is a fair approximation for short takes of continuous singing. Because the lyrics corpus is small (4.7 hours, against 23 hours of captioned music), each dataset receives a sampling weight: a weight of $w$ makes each of its songs $w$ times as likely to be drawn.

## 6.3 Testing whether the lyrics are used

A model can lower its loss on the lyrics corpus simply by learning how those particular songs sound, without reading their words. We therefore test the use of the lyrics directly, by giving the model the same audio with the correct words, with no words, and with words from another song.

**LYRICS-TEST**$(\theta, \mathcal{D}, B)$

*Input:* a trained model with parameters $\theta$, a set $\mathcal{D}$ of held-out songs with timed lyrics, and a number of batches $B$, each a few windows drawn at random.
*Output:* the mean paired difference $\bar{\Delta}$ and its standard error $s$, both in nats per code, and whether the difference is clearly positive.

```
for b ∈ [0, B) {
    draw a batch of windows from the songs in D
    L_b^right = loss of θ with each window's own lyrics
    L_b^none  = loss of θ with empty lyrics
```

5 $\quad\mathcal{L}_b^{\text{wrong}}$ = loss of $\theta$ with lines from another song
6 }
7 $\Delta_b = \mathcal{L}_b^{\text{wrong}} - \mathcal{L}_b^{\text{right}}$ for each $b$
8 $\bar{\Delta} = \frac{1}{B}\sum_b \Delta_b$
9 $s = \sqrt{\text{Var}(\Delta)/B}$
10 **return** $\bar{\Delta}$, $s$, and whether $\bar{\Delta} > 2s$

$\mathcal{L}_b$ denotes the loss of batch $b$, $\text{Var}(\Delta)$ is the sample variance of the $B$ differences, and the standard error $s$ estimates how much $\bar{\Delta}$ would vary if the test were repeated on other batches. Requiring $\bar{\Delta} > 2s$ corresponds roughly to 95 % confidence that the difference is positive. Lines 3–5 score the same windows, so the differences in line 7 compare like with like, and most of the variation between windows cancels. A model that ignores the words gives $\bar{\Delta} \approx 0$. A model that reads them should predict the audio better with the right words than with the wrong ones, giving $\bar{\Delta} > 0$, measured in nats per code. The wrong lines in line 5 always begin at a sung line rather than at silence, and comparing lines 4 and 5 helps distinguish a reaction to the presence of text from a reaction to its content.

# 7 Generation

Generation follows the factorization of Section 5.1 forward. Given the text, the model predicts a distribution over the codes at the next position, one code is drawn from each of these distributions, the drawn codes are appended to the sequence, and the step repeats until the piece reaches its requested length. Two refinements make this loop practical: a cache that avoids recomputing the past, and sampling rules that shape each draw.

## 7.1 The key/value cache

Position $p$ attends to the keys and values of positions $0, \ldots, p$. These do not change once they have been computed, because no position attends forward, so a cache can keep them [25], and each new position then costs one pass through the blocks instead of a pass over the whole sequence. In the procedure below, $j$ indexes the blocks, $q$, $k$ and $v$ are the query, key and value vectors of position $p$, and $\pi$ is the position map of Section 6.

**DECODE-STEP**$(z, p, \mathit{cache})$

*Input:* the symbols $z$ to place at position $p$ (a byte, or the $C$ codes of a step), the position index $p$, and the cache: for every block, the key and value vectors of positions $0, \ldots, p-1$.
*Output:* for each codebook, $K$ scores for the code at position $p+1$; the cache is extended by position $p$.

1 $h = \text{embed}(z) + \text{position}(\pi(p))$
2 **for** each block $j$ {
3 $\quad q, k, v$ = query, key and value projections of $\text{Norm}(h)$
4 $\quad \mathit{cache}_j[p] = (k, v)$
5 $\quad h = h + \text{Attend}(q,\ \mathit{cache}_j[0\,..\,p])$
6 $\quad h = h + \text{FeedForward}(\text{Norm}(h))$
7 }
8 **return** the output-head scores of $\text{Norm}(h)$

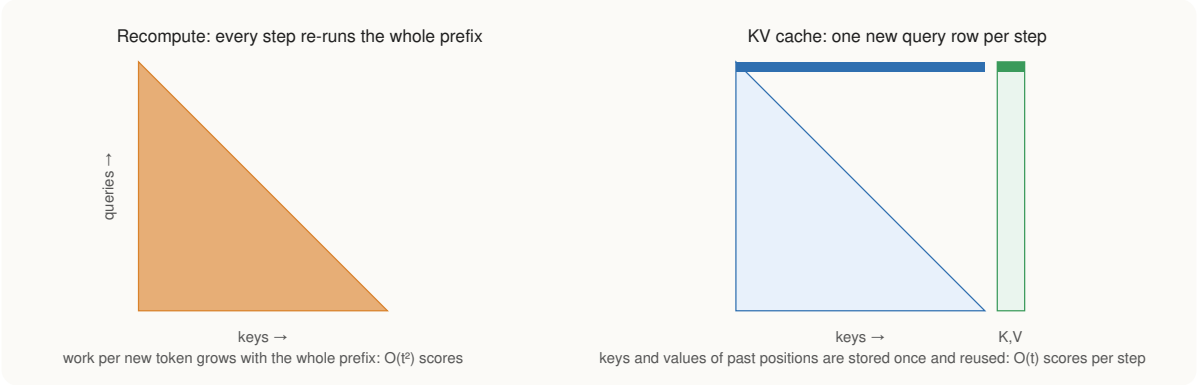



Figure 9: Why a key/value cache helps. Recomputing the whole prefix at every step evaluates a growing triangle of attention scores (left), so the work per new code grows with the length of the sequence; with a cache, each step computes one new row of scores against the stored keys and values (right).

**Theorem 7.1.** *Feeding a sequence position by position through* ***DECODE-STEP*** *yields, at every position, the same scores as a single forward pass over the whole sequence.*

*Proof.* We argue by induction over the blocks. At block 0, the input at position $p$ depends only on the token and the position of $p$. Suppose that the inputs of block $j$ agree at all positions up to $p$. Then the keys and values computed in line 3 agree, and when position $p$ is processed the cache holds exactly those of positions $0, \ldots, p$. In the full pass the mask gives zero weight to later positions, so attention at $p$ combines the same keys and values with the same weights, and the feed-forward network acts on each position separately. The outputs of block $j$, which are the inputs of block $j+1$, therefore agree up to position $p$. □

A test checks the theorem numerically, to within $10^{-4}$ at every position, for models with and without lyrics; the second case also checks the position map $\pi$. Because the cache has a fixed capacity, a step allocates no memory.

## 7.2 Sampling and guidance

For each codebook we keep the $k = 250$ most probable of the $K = 1\,024$ codes and draw among them in proportion to their probabilities [10]; excluding the unlikely codes avoids rare mistakes that the model could not recover from. Because the text was removed from 10 % of the training examples, the same model can predict both with the text, giving scores $z^{\text{text}}$, and without it, giving $z^{\text{none}}$; here $z$ denotes a vector of $K$ scores, which a softmax turns into probabilities. Classifier-free guidance [13] amplifies their difference:

$$z = z^{\text{none}} + s\,(z^{\text{text}} - z^{\text{none}}),$$

where $s$ is the guidance scale, a plain number ($s = 1$ uses the text as trained, and larger values follow it more strictly); this $s$ is unrelated to the standard error of Section 6.3. For instance, if two codes score $(2, 1)$ with the text and $(1, 1)$ without it, the first has probability 0.73; with $s = 3$ the scores become $(4, 1)$, and its probability rises to 0.95. We use $s = 3$, which doubles the computation per step. Lyrics longer than $L$ bytes are cut at line breaks into windows that are presented in turn, each over an equal share of the output, while the most recent audio is kept as context.

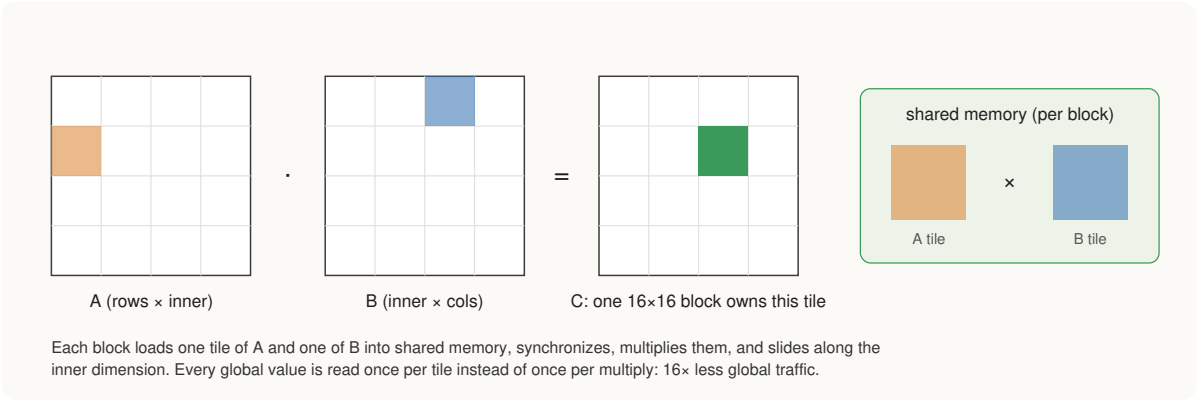



Figure 10: Shared-memory tiling, the starting point of register blocking. Each block of threads copies one tile of $A$ and one of $B$ into the fast memory it shares, synchronizes, multiplies them, and slides along the inner dimension, so every value fetched from global memory is used 16 times.

# 8 GPU kernels

Almost all of the arithmetic in StaleyNET is matrix multiplication: the spectrogram, the nearest-prototype search, every projection of the Transformer and both products inside attention. A GPU runs thousands of lightweight *threads* at once, grouped into *blocks* that cooperate, so the speed of the whole system depends on how well one matrix-product kernel keeps those threads busy and fed with data. Our tutorial [23] derives the shared-memory tiled product in detail. Here we describe one further step, register blocking, which roughly doubles its speed, and a way to make one gradient repeatable bit for bit.

## 8.1 Register blocking

A GPU runs many *threads* at once, grouped into *blocks* that cooperate. Each thread has up to 255 fast *registers*, the threads of a block share a small on-chip *shared memory*, and all threads can reach the large but slow *global memory*. In the tiled kernel for $C = AB$, a block of $16 \times 16$ threads copies square pieces (*tiles*) of $A$ and $B$ into shared memory, which reduces traffic to global memory sixteenfold [23, 14]. Every multiply-add still reads two values from shared memory, however, and shared memory is slower than a thread's registers. Register blocking [32] lets each thread compute an $r \times r$ group of outputs, so that every value it reads is used $r$ times (Figure 11).

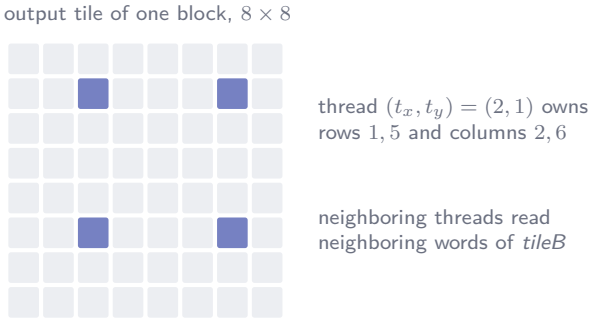



Figure 11: Register blocking with $4 \times 4$ threads and $r = 2$. Each thread computes $r^2$ outputs spread over the tile, in rows $t_y + 4i$ and columns $t_x + 4j$. Our kernel uses $16 \times 16$ threads, $r = 4$ and a $64 \times 64$ tile.

**BLOCKED-THREAD**$(t_x, t_y)$

*Input:* the column $t_x$ and row $t_y$ of a thread within a block of $16 \times 16$ threads, both in $[0, 16)$; the block computes a $64 \times 64$ tile of $C = AB$, and each thread computes $r \times r = 4 \times 4$ of its entries.
*Output:* the 16 entries of the tile owned by the thread, written to $C$.

```
1   sum[i][j] = 0 for i, j ∈ [0, r)
2   for each phase t {
3     copy a 64 × 16 tile of A into tileA
4     copy a 16 × 64 tile of B into tileB
5     BARRIER()
6     for u ∈ [0, 16) {
7       α_i = tileA[t_y + 16i][u] for i ∈ [0, r)
8       β_j = tileB[u][t_x + 16j] for j ∈ [0, r)
9       sum[i][j] = sum[i][j] + α_i β_j for i, j ∈ [0, r)
10    }
11    BARRIER()
12  }
13  store sum[i][j] at row t_y + 16i, column t_x + 16j of the tile
```

**BARRIER**() waits until every thread of the block has reached it. A *phase* handles 16 consecutive values of the inner dimension, so the loop of line 2 runs once for every 16 columns of $A$. *tileA* and *tileB* live in shared memory, while *sum*, $\alpha$ and $\beta$ live in the thread's registers; in this subsection $\alpha$ and $\beta$ are these register copies, unrelated to the momentum and the Kaiser parameter above. Lines 7 and 8 read $2r = 8$ values from shared memory, and line 9 performs $r^2 = 16$ multiply-adds, so each multiply-add costs $2/r = 0.5$ reads instead of 2. Global traffic falls as well: each phase copies $2 \cdot 64 \cdot 16$ values for $64 \cdot 64 \cdot 16$ multiply-adds, so the product of an $M \times K$ and a $K \times N$ matrix (here $M$, $N$ and $K$ are matrix dimensions) fetches

$$\frac{M}{64} \cdot \frac{N}{64} \cdot \frac{K}{16} \cdot 2 \cdot 64 \cdot 16 = \frac{2MNK}{64}$$

values of 4 bytes each, a quarter of the $2MNK/16$ needed with $16 \times 16$ tiles. The two barriers play the same roles as in the tiled kernel, whose correctness is proved in [23]. On the Orin NX the blocked kernel is 1.8 to 2.3 times faster than the tiled one (Table 9).

## 8.2 A repeatable embedding gradient

The embedding table has one row of $d$ numbers per vocabulary symbol, and its gradient adds the gradient vector $g_p$ (also $d$ numbers) of each position $p$ into the row of the token $z_p$ that the position holds. Many positions share a token. With atomic additions, threads would add into a shared row in an order that varies from run to run, and because floating-point addition is not associative, the result would vary as well. We fix the order with a counting sort.

**EMBEDDING-GRADIENT**$(z, g, V)$

*Input:* for every position $p$, its token $z_p \in [0, V)$ and its gradient vector $g_p$ of $d$ numbers; and the number of rows $V$ of the embedding table (the vocabulary size).
*Output:* the gradient $G$ of the embedding table, a $V \times d$ matrix with one row per token; $c \in [0, d)$ indexes its columns.

```
count[v] = |{p : z_p = v}| for v ∈ [0, V)  // on the CPU
start[v] = Σ_{u<v} count[u]
list = positions sorted by token, then by position
for each token v and column c, in parallel {  // on the GPU
    G[v][c] = Σ g_p[c] over p in list[start[v] .. start[v + 1]), in order
}
```

For the tokens $z = (2, 0, 2, 1)$, the counts are $(1, 1, 2)$, the starts are $(0, 1, 2, 4)$ and the list is $(1, 3, 0, 2)$, so token 2 receives $g_0 + g_2$ in that order. Each entry is summed by one thread in a fixed order, so the result is the same on every run, and a whole training run can be repeated bit for bit on the same device.

# 9 Experiments

Our main corpus is the Song Describer Dataset [21]: 706 two-minute Creative Commons excerpts with 1 106 captions written by listeners. Songs are split between training and evaluation by artist, so that no voice appears on both sides, leaving 667 songs for training and 39 for evaluation; Table 8 in the appendix describes the hardware. We report spectral errors as the mean absolute difference between two log-mel spectrograms, averaged over frames and bands, in *log units per band*. Because the spectrogram holds natural logarithms of energy, an error of one log unit means that a band's energy is off by a factor of $e \approx 2.7$, or $10 \log_{10} e \approx 4.3$ dB.

**Correctness and speed.** All 169 tests pass on the Orin NX, and the kernel tests also pass on the Thor. They compare every kernel with a CPU implementation at deliberately awkward sizes, check every gradient numerically, verify the cache theorem, and confirm that a small network can memorize a fixed batch. On the Orin NX, register blocking makes a training step of the first model 1.8 times faster than tiling and 9.3 times faster than the direct kernel (Table 10).

**First training runs.** Table 4 summarizes the runs with the first tokenizer. A training *step* here is one update of the parameters, computed from a batch of 8 sequences; the held-out loss is measured on songs by artists that the model never heard during training. A run of 1 500 steps reached a held-out loss of 4.93, and a run of 15 000 steps on the Thor reached 4.02. When we restricted the data to the 281 songs whose captions mention a voice, a model fine-tuned from the longer run reached its best held-out loss of 4.23 after only 250 steps, after which its held-out loss rose while its training loss continued to fall; a model trained from scratch on the same songs reached 4.72. The vocal subset appears to be too small for long training.

Table 4: Best held-out loss with the first tokenizer, in nats per code (uniform guess: 6.93).

| run | data | steps | loss (nats per code) |
|---|---|---|---|
| first run, Orin NX | all songs | 1 500 | 4.93 |
| longer run, Thor | all songs | 15 000 | 4.02 |
| fine-tuned, Thor | vocal songs | 250 | 4.23 |
| from scratch, Orin NX | vocal songs | 5 500 | 4.72 |

### 9.1 What limits the sound

The generated voices sounded pitched but blurred. To find out why, without involving the model, we rendered 12-second excerpts of four held-out vocal songs through each stage of the pipeline and compared the results with their exact spectrograms (Figure 12 and Table 5).

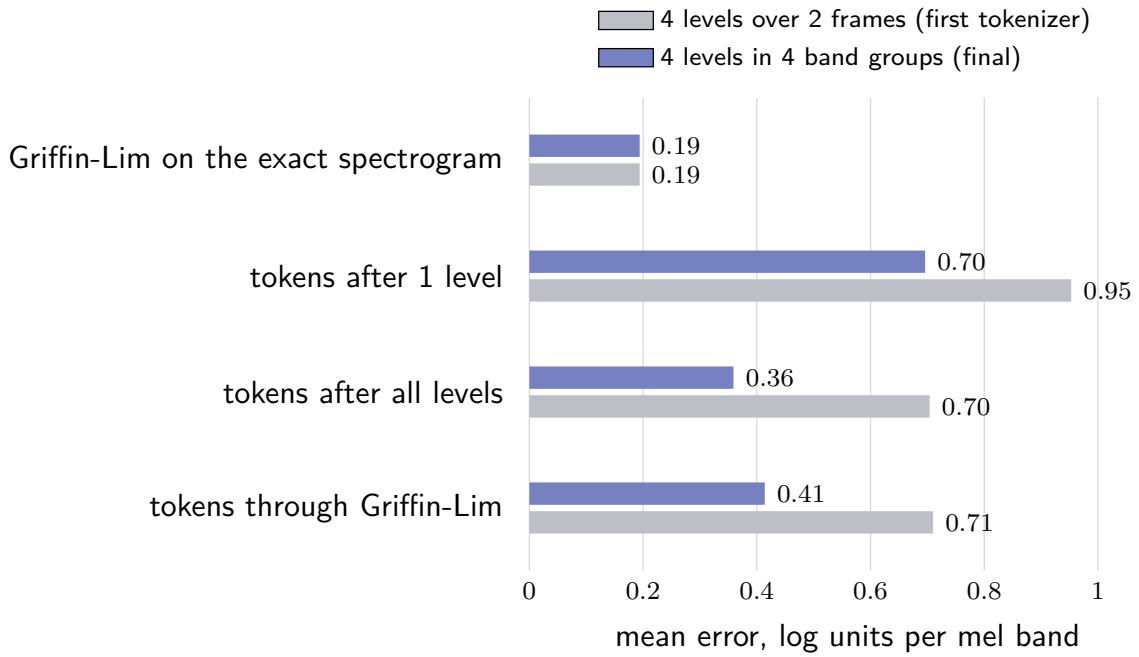



Figure 12: Mean spectral error of each stage on four held-out vocal excerpts, for the first tokenizer (gray) and the final one (blue). The bottom pair is the best that any model trained on these tokens could achieve.

Table 5: Spectral error of each stage on four held-out vocal excerpts, in log units per mel band, for the first tokenizer and the final one.

| | | first tokenizer | | final tokenizer | |
|---|---|---|---|---|---|
| excerpt | vocoder | tokens | end to end | tokens | end to end |
| guitar, brass, female voice | 0.230 | 0.807 | 0.821 | 0.425 | 0.480 |
| folk metal, male voice | 0.152 | 0.707 | 0.703 | 0.353 | 0.378 |
| electronic, vocal sample | 0.170 | 0.569 | 0.554 | 0.268 | 0.338 |
| bass, piano, wordless voice | 0.222 | 0.732 | 0.763 | 0.390 | 0.460 |
| mean | 0.194 | 0.704 | 0.710 | 0.359 | 0.414 |

With the first tokenizer, the tokens alone caused 3.6 times the error of the vocoder, and the end-to-end error of a perfect model, 0.71, was almost entirely due to the tokenizer. An error of about 0.7 log units, roughly 3 dB in every band, is enough to blur the harmonic and formant detail on which an intelligible voice depends. This measure does not capture the phase artifacts of Griffin-Lim, which remain audible, so the tokenizer is the main spectral limitation rather than the only audible one.

To examine the model on its own, we trained it on a single 12-second excerpt of 562 steps. Its training loss fell to 0.054 after 100 steps and to 0.002 after 200. We then generated 12 seconds from the excerpt's caption and compared the generated codes with the excerpt's own (Table 6). Greedy decoding reproduced all 562 steps, and top-$k$ sampling departed from the excerpt only at isolated codes, without drifting away.

Table 6: Generated codes compared with a memorized excerpt.

| decoding | agreement per codebook (%) | identical prefix (steps) |
|---|---|---|
| greedy | 100.0, 100.0, 100.0, 100.0 | 562 of 562 |
| top-$k$, seed 1 | 99.6, 100.0, 100.0, 99.6 | 26 |
| top-$k$, seed 2 | 100.0, 99.8, 99.3, 99.8 | 26 |

The model can reproduce audio exactly, and when it does, the result sounds like the tokenizer's reconstruction. These two observations led us to improve the tokens before training larger models.

### 9.2 A finer tokenizer

Guided by this analysis, we compared tokenizers that differ in the number of frames per step, the number of band groups and the number of levels (Table 12). Quantizing one frame per step in 4 band groups with 4 levels each lowered the mean error on the held-out songs from 0.776 to 0.401. A control without band groups suggests that most of this gain comes from the larger number of codes per step, and a smaller part from the grouping. On the four vocal excerpts of Figure 12, the error of the tokens fell from 0.70 to 0.36, and the end-to-end error from 0.71 to 0.41. The price is a vocabulary of 16 codebooks instead of 4.

### 9.3 A larger model on the finer tokens

With the finer tokens we began training a larger model on the Thor: 8 blocks of width 512, 42.9 million parameters, with the lyrics input enabled from the start. It is trained on all 742 training songs of the combined corpora, with the lyrics corpus weighted 4-fold and two takes recorded by the first author weighted 10-fold, in batches of 8 sequences of 1 408 positions. A step takes 2.0 seconds and about 11 GB of the Thor's 128 GB of memory. Training is still in progress; after 8 000 of 20 000 steps the held-out loss is 5.11 nats per code, averaged over its 16 codebooks, and still falling. Because the vocabulary differs, this figure is not comparable with those of Table 4.

### 9.4 Lyrics

We extended both vocal models of Table 4 with a lyrics field and trained them further on vocal songs, JamendoLyrics, and songs whose captions mention rock or a related style. Table 7 gives the result of **Lyrics-Test** for a model trained on the Thor with the lyrics corpus weighted 8-fold, over 48 batches of 4 windows.

Table 7: Loss with the right lyrics, no lyrics and the wrong lyrics, and the paired difference $\bar{\Delta} \pm s$.

| songs | right | none | wrong | $\bar{\Delta} \pm s$ |
|---|---|---|---|---|
| | | (nats per code) | | |
| held out (6) | 4.3345 | 4.3409 | 4.3345 | $+0.0000 \pm 0.0004$ |
| training (73) | 4.0337 | 4.0395 | 4.0378 | $+0.0042 \pm 0.0006$ |

On the held-out songs, the right and the wrong lyrics give the same loss, so this model does not yet use the words. On the training songs the difference is statistically clear but small, which

we take to indicate memorization of those songs rather than a learned relation between words and sound. We hope that the paired test will be useful as a clear criterion for future attempts.

## 10 Limitations

The tokenizer, although improved, still limits the fidelity of the output, and Griffin-Lim estimates phase without ever having heard real audio, which leaves a metallic smear that is most audible on voices (Section 3). The learned vocoder of Section 3.5 is meant to remove it, but it is trained with spectral losses only; without the adversarial losses of Vocos [27], such vocoders tend to sound slightly muffled in the highest frequencies, and adding discriminators is our next step. The data contain comparatively little singing (298 of the 706 songs mention a voice) and very little of some styles (two songs are described as grunge). Our models are much smaller than published systems [3], and we have not yet carried out a formal listening test, so we make no claims about perceived quality. On the implementation side, all arithmetic uses 32-bit floating point without tensor cores, and attention stores its full matrix of scores [5].

## 11 Conclusion

StaleyNET models music as a sequence of tokens whose probability given a text is factorized by the chain rule, learns every factor with one autoregressive Transformer, and generates by sampling the factors in turn. Building each stage ourselves, in Rust and on embedded GPUs, made it possible to measure each one in isolation. Those measurements located the main limitation on sound quality in the tokenizer rather than in the language model, and a tokenizer that quantizes groups of frequency bands with more codes per step halved the reconstruction error; a larger model is now training on it. We hope the worked examples, the stage-by-stage analysis and the paired lyrics test will help others who build similar systems on modest hardware.

## References


[1] Andrea Agostinelli et al. MusicLM: Generating music from text, 2023. arXiv:2301.11325.

[2] Zalán Borsos et al. AudioLM: A language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023. arXiv:2209.03143.

[3] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.05284.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2205.14135.

[6] Michaël Defferrard, Kirell Benzi, Pierre Vandergheynst, and Xavier Bresson. FMA: A dataset for music analysis. In *Proceedings of the International Society for Music Information Retrieval Conference*, 2017. arXiv:1612.01840.

[7] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2023. arXiv:2210.13438.

[8] Prafulla Dhariwal, Heewoo Jun, Christine Payne, Jong Wook Kim, Alec Radford, and Ilya Sutskever. Jukebox: A generative model for music, 2020. arXiv:2005.00341.

[9] Simon Durand, Daniel Stoller, and Sebastian Ewert. Similarity-based audio-lyrics alignment of multiple languages. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2023. arXiv:2306.07744.

[10] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Annual Meeting of the Association for Computational Linguistics*, 2018. arXiv:1805.04833.

[11] Daniel W. Griffin and Jae S. Lim. Signal estimation from modified short-time Fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(2):236–243, 1984.

[12] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs), 2016. arXiv:1606.08415.

[13] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021. arXiv:2207.12598.

[14] David B. Kirk, Wen mei W. Hwu, and Izzat El Hajj. *Programming Massively Parallel Processors*. Morgan Kaufmann, 4 edition, 2022.

[15] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. In *Advances in Neural Information Processing Systems*, 2020. arXiv:2010.05646.

[16] Rithesh Kumar, Prem Seetharaman, Alejandro Luebs, Ishaan Kumar, and Kundan Kumar. High-fidelity audio compression with improved RVQGAN. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.06546.

[17] Jinglin Liu, Chengxi Li, Yi Ren, Feiyang Chen, and Zhou Zhao. DiffSinger: Singing voice synthesis via shallow diffusion mechanism. In *AAAI Conference on Artificial Intelligence*, 2022. arXiv:2105.02446.

[18] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. arXiv:2201.03545.

[19] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[20] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. arXiv:1711.05101.

[21] Ilaria Manco, Benno Weck, SeungHeon Doh, Minz Won, Yixiao Zhang, Dmitry Bogdanov, Yusong Wu, Ke Chen, Philip Tovstogan, Emmanouil Benetos, Elio Quinton, György Fazekas, and Juhan Nam. The song describer dataset: a corpus of audio captions for music-and-language evaluation. In *NeurIPS Workshop on Machine Learning for Audio*, 2023. arXiv:2311.10057.

[22] NVIDIA. cuda-oxide: A Rust to PTX compiler backend. https://github.com/NVlabs/cuda-oxide, 2026.

[23] Arpan Pathak. An introduction to tiled parallel programming, simplified, 2026. Manuscript.

[24] Nathanaël Perraudin, Peter Balazs, and Peter L. Søndergaard. A fast Griffin-Lim algorithm. In *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, 2013.

[25] Reiner Pope et al. Efficiently scaling transformer inference. In *Conference on Machine Learning and Systems*, 2023. arXiv:2211.05102.

[26] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[27] Hubert Siuzdak. Vocos: Closing the gap between time-domain and Fourier-based neural vocoders for high-quality audio synthesis, 2023. arXiv:2306.00814.

[28] Malcolm Slaney. Auditory toolbox, version 2. Technical Report 1998-010, Interval Research Corporation, 1998.

[29] Julius O. Smith. Digital audio resampling home page. Center for Computer Research in Music and Acoustics, Stanford University, 2002. https://ccrma.stanford.edu/~jos/resample/.

[30] Daniel Stoller, Simon Durand, and Sebastian Ewert. End-to-end lyrics alignment for polyphonic music using an audio-to-character recognition model. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2019. arXiv:1902.06797.

[31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. arXiv:1706.03762.

[32] Vasily Volkov and James W. Demmel. Benchmarking GPUs to tune dense linear algebra. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2008.

[33] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, 2020. arXiv:2002.04745.

[34] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022. arXiv:2105.13626.

[35] Ryuichi Yamamoto, Eunwoo Song, and Jae-Min Kim. Parallel WaveGAN: A fast waveform generation model based on generative adversarial networks with multi-resolution spectrogram. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2020. arXiv:1910.11480.

[36] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021. arXiv:2107.03312.

[37] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, 2019. arXiv:1910.07467.

Table 8: The two devices. Core counts and memory bandwidth are the manufacturer's specifications; memory capacity, compute capability and software versions were read from the devices.

| | Jetson Orin NX 16 GB | Jetson AGX Thor |
|---|---|---|
| GPU | NVIDIA Ampere, 1 024 CUDA cores, 32 tensor cores | NVIDIA Blackwell, 2 560 CUDA cores, 96 tensor cores |
| compute capability | 8.7 | 11.0 |
| CPU | 8-core Arm Cortex-A78AE | 14-core Arm Neoverse-V3AE |
| memory | 16 GB LPDDR5, shared by CPU and GPU | 128 GB LPDDR5X, shared by CPU and GPU |
| memory bandwidth | 102 GB/s | 273 GB/s |
| system software | L4T R36.4.3, CUDA 12.6 | L4T R39.2.1, CUDA 13.2 |
| Rust toolchain | nightly-2026-04-03 | nightly-2026-10-11 |
| cuda-oxide | revision b7fe543 | revision e027ac3 |

# A Hardware and software

Both devices were shared with other work during our experiments. In particular, language-model services occupied most of the Thor's memory until the larger training run began, at which point they were stopped.

# B Benchmarks

All figures in this appendix were measured on the Orin NX with `staleynet bench`. Each timing is the mean of five runs after one warm-up run, and each table entry is the median of three such measurements, because the clock frequency of the device varies noticeably from run to run.

Table 9: Matrix products at the sizes used by the first model, in GFLOP/s.

| product ($M \times N \times K$) | direct | tiled | register-blocked | blocked / tiled |
|---|---|---|---|---|
| attention projections ($2048 \times 768 \times 256$) | 46.8 | 258.6 | 579.0 | 2.2 |
| feed-forward ($2048 \times 1024 \times 256$) | 55.3 | 255.5 | 450.6 | 1.8 |
| output heads ($2048 \times 4096 \times 256$) | 31.2 | 222.1 | 458.5 | 2.1 |
| spectrogram ($4096 \times 513 \times 1024$) | 47.7 | 236.1 | 541.7 | 2.3 |

Table 10: One training step of the first model on $4 \times 512$ positions, forward and backward.

| matrix-product kernel | time per step (ms) |
|---|---|
| direct | 2 930 |
| tiled | 576 |
| register-blocked | 315 |

On the Thor, measured during training runs of identical shape, register blocking reduced a training step of the first model on $4 \times 768$ positions from 238 ms to 147 ms, and a step of the larger model on $8 \times 1\,408$ positions takes about 2.0 seconds.

Table 11: Other measurements on the Orin NX.

| measurement | result |
|---|---|
| log-mel spectrogram of 60 s, GPU | 47 ms |
| log-mel spectrogram of 60 s, CPU (rustfft, 8 threads) | 73 ms |
| decoding step, key/value cache | 6.2 ms |
| decoding step, recomputing the context | 17.1 ms |

# C Tokenizer study

Table 12 lists the mean reconstruction error over all 39 held-out songs for the tokenizers we compared, each trained on the same training songs with 1 024 prototypes per codebook. The columns give the number of frames per step, the number of band groups, the number of residual levels in each group, and the resulting number of codes per step.

Table 12: Held-out reconstruction error of different tokenizers, in log units per mel band (one log unit is a factor of $e$ in energy, about 4.3 dB). A frame covers 10.7 ms of audio; every codebook has 1 024 prototypes.

| frames | groups | levels | codes | error (log units per band) |
|---|---|---|---|---|
| 2 | 1 | 4 | 4 | 0.776 |
| 2 | 1 | 8 | 8 | 0.643 |
| 1 | 1 | 4 | 4 | 0.745 |
| 1 | 1 | 8 | 8 | 0.588 |
| 1 | 4 | 2 | 8 | 0.597 |
| 1 | 8 | 1 | 8 | 0.612 |
| 1 | 1 | 16 | 16 | 0.427 |
| 1 | 8 | 2 | 16 | 0.403 |
| 1 | 4 | 4 | 16 | 0.401 |

At 8 codes per step, dividing the bands into groups did not help. With 16 codes per step it helped a little: 16 levels without groups reached 0.427, and 4 groups of 4 levels reached 0.401. Most of the improvement over the first tokenizer therefore comes from describing each step with more codes, and a smaller part, about 6 %, from quantizing the band groups separately.